

## 重点回答

### 问题 1: SGLang 的 RadixAttention / RadixCache 解决什么问题?

答: 在大模型在线推理服务中, 大量用户请求经常会出现重复、相似的 Prompt 前缀。传统 KV 缓存机制中, 每一条请求都会独立计算并保存属于自己的 KV 缓存, 即便多条请求拥有完全一致的前缀文本, 依旧会重复执行 Prefill 计算, 同时多份相同 KV 副本会大量占用 GPU 显存, 造成算力浪费与显存开销上涨, 还会抬高 TTFT 首 token 延迟。

RadixAttention 与 RadixCache 就是为了解决上述问题。它基于前缀树 (Radix Tree) 对 KV 缓存做组织管理, 实现 Prefix Reuse 前缀复用。当新请求到来时, 如果该请求的 prompt 前缀已经在缓存树中存在, 就可以直接复用已经计算完成的 KV 缓存, 不再重复对公共前缀执行 Prefill 前向计算。一方面减少 GPU 的计算量, 降低 TTFT; 另一方面多个请求共享同一份 KV 缓存副本, 节约宝贵的 GPU 显存, 提升系统整体并发吞吐能力。

### 问题 2: SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分?

答: page-sized KV cache 是底层存储管理机制, 贯穿 Prefill 阶段和 Decode 迭代阶段。系统会把 KV 缓存分割成固定大小的缓存页, 负责缓存页的分配、写入、回收、复用, 为推理过程提供 KV 数据存储, 是缓存的硬件存储基础。

prefix reuse 即前缀复用, 主要发生在 Prefill 阶段。新请求进入系统, 在执行 Prefill 计算之前, 会查询 RadixCache 前缀树; 若匹配到已存在的公共前缀 KV 缓存, 就直接复用该部分缓存, 仅对请求中新的、未缓存的 token 执行 Prefill 计算。Prefix Reuse 是建立在 page-sized KV cache 之上的高层优化逻辑。

### 问题 3: vLLM 的 PagedAttention 解决什么问题? 它和 SGLang 的 RadixAttention /paged KV 机制有什么联系与差异?

答: vLLM 提出的 PagedAttention 主要解决传统连续 KV 缓存带来的显存内存碎片化问题。传统方案中每个请求占用一块连续显存空间, 请求结束释放后会产生大量零散空闲显存块, 显存利用率低, 限制并发请求数量。PagedAttention 借鉴操作系统虚拟内存分页思想, 将 KV 缓存切分为固定大小页面, 页面不需要物理连续, 可以分散存放, 实现页面动态分配与回收, 极大缓解显存碎片, 提升推理服务的最大并发与吞吐。

## 二者联系

RadixAttention (SGLang) 和 PagedAttention (vLLM) 底层都采用分页 KV 缓存的设计思路, 都把 KV 切分为固定大小页面, 依靠分页机制缓解显存碎片, 提升多请求批处理并发性能。

## 二者差异

1. PagedAttention 仅管理单条请求内部的缓存页面，**不支持跨请求之间共享 KV 缓存**，不同请求的 KV 页面完全相互独立，即便请求存在相同前缀，也要分别计算、分别存储。
2. RadixAttention 继承分页 KV 的基础能力，额外引入 Radix 前缀树索引，增加跨请求 Prefix Reuse 前缀复用功能，多条请求可以共享公共 prompt 前缀的 KV 缓存，减少 Prefill 计算开销，进一步压低 TTFT。
3. 从能力层级看，PagedAttention 解决显存碎片；RadixAttention 在解决碎片之外，额外实现跨请求缓存共享优化。

**比喻：**PagedAttention 可以同时做多杯奶茶，但是每一杯的基底都要单独煮；

RadixCache 在它基础上，发现多杯奶茶基底相同时，基底只煮一次，大家共用基底，只单独做各自不同的配料。

- **PagedAttention (vLLM)** 奶茶店可以同时接待很多客人（支持并发并行）。它把每一杯奶茶的原料拆成独立小份分装（KV 缓存分页），解决原料存放空间碎片化的问题，能接更多订单。但是**每位客人的奶茶底单独煮**。就算两位客人都点【红茶底 + 冰】，只是配料不同，也要分别煮两份红茶。A 的茶底只能 A 用，B 的茶底只能 B 用，茶底不能共用。
- **RadixCache (SGLang RadixAttention, 基于分页缓存升级)** 同样可以同时接待很多客人，也保留原料分块存放的优点。店员会检查订单，发现多个订单拥有相同的前置要求，就**一次性煮一份公共红茶底**，这份茶底可以给所有匹配前缀的客人共用。只需要给每个人单独制作后面不一样的配料。公共茶底只需要制作一次，节省时间，顾客拿到第一口奶茶的等待时间（TTFT）变短。每位客人最后的配料和成品奶茶依旧独立分开，不会混单。